



Boletín 01 Intermedio — Algorítmica y Pseudocódigo

10 retos algorítmicos aplicados que combinan bucles centinela, validaciones robustas con Repetir, modularidad, recursividad, primos, descomposición de dígitos, años bisiestos y arquitectura del software.

Este boletín contiene **10 ejercicios de nivel intermedio** para poner a prueba tu capacidad de estructurar algoritmos modulares, robustos y eficientes en pseudocódigo. Intenta resolverlos antes de consultar el [Boletín Intermedio Resuelto](#).



Descarga en PDF

Puedes [descargar este boletín intermedio en PDF](#) para imprimirlo o trabajar sin conexión. También dispones del [PDF con las soluciones completas](#).

Ejercicio 1: Validación estricta de entrada con bucle Repetir

Diseña un algoritmo en pseudocódigo que solicite al usuario una calificación académica. El programa debe obligar al usuario a repetir la introducción de la nota mientras el valor introducido esté fuera del rango válido `[0.0, 10.0]`. Una vez introducida una nota correcta, el algoritmo debe clasificarla e imprimir su calificación cualitativa:

- `[0.0, 5.0]` : Suspenso
- `[5.0, 6.0]` : Aprobado
- `[6.0, 7.0]` : Bien
- `[7.0, 9.0]` : Notable
- `[9.0, 10.0]` : Sobresaliente

Ejercicio 2: Bucle con centinela: Contador de pares e impares

Diseña un algoritmo que solicite números enteros al usuario de forma continua hasta que introduzca el número `0` (**valor centinela de parada**). Al terminar la introducción de datos, el programa debe mostrar por pantalla:

- Cuántos números introducidos eran **pares**.
- Cuántos números introducidos eran **impares**.

(Nota: El número 0 no debe contabilizarse como dato a evaluar, solo marca el final de la secuencia).

Ejercicio 3: Búsqueda de mayor y menor de 5 números sin arrays

Diseña un algoritmo que pida 5 números enteros cualesquiera por teclado y determine cuál es el **mayor** y cuál es el **menor** de todos ellos, utilizando únicamente variables escalares simples y una estructura `Para` (sin utilizar arrays ni vectores).

- Trampa didáctica a evitar:** ¿Por qué inicializar las variables `mayor` y `menor` a cero al principio es un grave error de lógica que fallará si el usuario introduce únicamente números negativos? ¿Cómo debe inicializarse correctamente?

Ejercicio 4: Subalgoritmo Factorial iterativo y análisis de traza

1. Diseña un subalgoritmo/función llamado `factorial(n)` que reciba un número entero positivo `n` y calcule su factorial ($n! = 1 \times 2 \times 3 \times \dots \times n$) utilizando una estructura iterativa `Para`.
2. Elabora la tabla de traza detallada paso a paso para la invocación `factorial(4)`, registrando la evolución en memoria de las variables en cada ciclo del bucle.

Ejercicio 5: Introducción a la recursividad (`factorialRecursivo`)

Un subalgoritmo puede invocarse a sí mismo para resolver un problema dividiéndolo en instancias más pequeñas; esta técnica se denomina **recursividad**:

1. Diseña una versión recursiva llamada `factorialRecursivo(n)`. Identifica con claridad cuál es el **caso base** (la condición que frena la recursión) y cuál es el **caso recursivo**.
2. Compara la solución iterativa del Ejercicio 4 con la versión recursiva: reflexiona sobre la legibilidad, la elegancia del código y el consumo de memoria en la pila (*stack*).

Ejercicio 6: Diseño modular integral: Calculadora de IMC con validación

Diseña una aplicación modular en pseudocódigo estructurada en funciones y procedimientos para calcular el **Índice de Masa Corporal (IMC = peso / altura²)**:

1. `SubAlgoritmo peso ← pedirPeso()`: Solicita el peso en kilogramos y valida con `Repetir ... Hasta Que` que sea estrictamente positivo (`> 0`).
2. `SubAlgoritmo altura ← pedirAltura()`: Solicita la altura en metros y valida con `Repetir ... Hasta Que` que sea un valor positivo (`> 0`).
3. Algoritmo principal que invoque a los subalgoritmos anteriores, calcule el IMC y clasifique el resultado según las categorías de la OMS:
 - `IMC < 18.5`: Bajo peso
 - `18.5 ≤ IMC < 25.0`: Peso normal / Saludable
 - `25.0 ≤ IMC < 30.0`: Sobrepeso
 - `IMC ≥ 30.0`: Obesidad

Ejercicio 7: Comprobador de números primos optimizado (`esPrimo`)

Un número entero mayor que 1 es **primo** si y solo si es divisible únicamente por 1 y por sí mismo. Diseña un subalgoritmo `resultado ← esPrimo(n)` que reciba un número entero `n` y devuelva `Verdadero` si es primo o `Falso` en caso contrario. Optimiza el bucle para que descarte números menores que 2 y no realice iteraciones redundantes.

Ejercicio 8: Contador y sumador de dígitos de un número entero

Diseña un algoritmo que solicite al usuario un número entero positivo cualquiera (por ejemplo, `4825`) y, mediante un bucle, calcule y muestre:

1. La cantidad total de dígitos que posee (en el ejemplo: `4` dígitos).
2. La suma de todos sus dígitos individuales (`4 + 8 + 2 + 5 = 19`).

Pista: Emplea sucesivas divisiones enteras entre `10` para recortar el último dígito y el operador módulo `MOD 10` para extraerlo.

Ejercicio 9: Determinación de Año Bisiesto (Regla Gregoriana)

En el calendario gregoriano, un año es **bisiesto** (tiene 366 días) si es divisible entre `4`, excepto si es divisible entre `100`, a menos que también sea divisible entre `400`. *Ejemplos:*

- El año `2024` es bisiesto (divisible por 4 y no por 100).
- El año `1900` **no** fue bisiesto (divisible por 100 pero no por 400).
- El año `2000` **sí** fue bisiesto (divisible por 400).

Diseña un algoritmo que pida un año al usuario y determine mediante una **única condición lógica combinada** (usando `MOD`, `Y`, `0`) si el año es bisiesto o común.

Ejercicio 10: Diagnóstico de errores y arquitectura de la JVM

Responde de forma razonada a las dos siguientes cuestiones teóricas de ingeniería:

1. **Tipología de errores:** Clasifica cada uno de los siguientes incidentes como error **sintáctico**, **semántico/lógico** o **en tiempo de ejecución (runtime)**, justificando cuándo y cómo se detecta:
 - (a) Olvidar una palabra reservada de cierre como `FinSi` o un punto y coma en el código.
 - (b) Calcular el área de un círculo utilizando por error la fórmula del perímetro (`2 × PI × radio`).
 - (c) Calcular la velocidad media dividiendo la distancia entre el tiempo cuando el usuario introduce un tiempo igual a `0`.
2. **Ventajas del modelo híbrido de Java:** Explica qué ventaja práctica fundamental ofrece el modelo de Bytecode + JVM de Java frente a un lenguaje puramente compilado como C, y qué ventaja ofrece frente a un lenguaje puramente interpretado como Python.